

HealthAI Coach — Soutenance de projet

7/10/2026, 1:43:41 PM

1/43 HealthAI Coach — Soutenance de projet

- Bonjour, je présente le backend data de HealthAI Coach, une plateforme de coaching santé
 - Équipe de 4 : répartition en 3 rôles (architecture/data, API, ETL)
 - Plan : contexte -> architecture -> ce qui a été construit -> démo live -> bilan
 - Durée cible ~50 min avec la démo
-

2/43 Sommaire

- Progression : POURQUOI (contexte) -> COMMENT (architecture, modèle, pipeline, API) -> PREUVE (démo live) -> BILAN
 - La démo live est le moment fort : on y verra les données réelles remplacer les données de démo en direct
-

3/43 01 • Contexte & objectifs

- Poser le décor avant de rentrer dans le technique : qui, pourquoi, pour qui
-

4/43 HealthAI Coach, c'est quoi ?

- Insister sur le périmètre : c'est un projet backend/data, pas une appli complète
 - Les captures qu'on montrera sont des outils internes (admin, Metabase), pas l'app finale
-

5/43 Modèle économique

- Le modèle économique n'est pas cosmétique : il a façonné le schéma (table subscriptions), les dépendances FastAPI (CurrentPremiumUser), et les règles d'accès
-

6/43 Répartition des rôles

- Workflow Git strict dès le départ : ça a payé, plusieurs bugs réels attrapés en revue avant fusion
 - Le rôle A a repris une partie de l'orchestration Airflow en fin de projet, faute de temps côté équipe ETL
-

7/43 02 • Architecture du système

- Transition : maintenant qu'on sait pourquoi, comment le système est structuré globalement
-

8/43 Vue d'ensemble

- L'ETL alimente Postgres, orchestré par Airflow
 - Trois consommateurs en aval : API pour les utilisateurs finaux, interface admin pour la qualité, Metabase pour le pilotage
 - On reverra ce schéma en vrai pendant la démo
-

9/43 Choix technologiques

- PostgreSQL plutôt que NoSQL : les contraintes d'intégrité (CHECK, FK) auraient dû être réimplémentées côté application dans un magasin sans schéma — pas un choix par défaut, un choix motivé
 - Alembic : outil de migration de schéma pour SQLAlchemy — chaque évolution (ex. ajout organizations/subscriptions, puis workout_plans/nutrition_plans) devient un script Python versionné et rejouable, avec l'historique suivi en base via la table alembic_version (plutôt que des ALTER TABLE manuels non tracés)
 - Gradio : compromis assumé, au prix de limitations d'accessibilité qu'on détaillera
-

10/43 03 • Modélisation des données

- Cœur du projet : tout le reste (ETL, API, admin) s'organise autour de ce schéma
-

11/43 Le schéma en un coup d'œil

- Le choix d'auto-documenter en SQL est un point fort qu'on assumera dans le bilan
-

12/43 Modèle Conceptuel de Données

- Cardinalités au format Merise (min,max)
 - USERS au centre, rattaché à 8 domaines : nutrition, activité, biométrie, médical, alimentaire, forme, recommandations, plans IA
 - Trait plein = FK obligatoire, pointillé = FK optionnelle (ex. resolved_by, nullable)
-

13/43 Conventions d'intégrité

- La contrainte CHECK a fait exactement son travail : bloquer une donnée invalide en base
 - Les tests unitaires mockaient l'entrée, donc ne touchaient jamais la vraie contrainte — c'est ce qui a motivé la discipline "toujours retester contre un vrai Postgres" pour la suite
-

14/43 Conventions d'intégrité (suite)

- Point à raconter : un bug réel où l'ETL produisait 'Other' (majuscule) au lieu de 'other', rejeté par PostgreSQL mais indétecté par les tests unitaires mockés — on y revient dans le bilan
-

15/43 04 · Pipeline ETL & orchestration Airflow

- Section la plus riche en obstacles réels rencontrés — bien la préparer
-

16/43 Structure du pipeline

- extract : télécharge les 4 datasets Kaggle + pagine l'API ExerciseDB
 - transform : renommage colonnes, typage, remplissage déterministe
 - load : upsert idempotent (staging + ON CONFLICT pour food_items/exercises, INSERT...ON CONFLICT...RETURNING pour users)
 - quality_check : règles réelles, journalise dans data_quality_log
-

17/43 Extraction multi-sources (E)

- Travail de Johane et William — la pagination par curseur sur ExerciseDB n'était pas documentée clairement côté API, découverte par tâtonnement
 - Le stockage brut intermédiaire sert de filet de sécurité : en cas d'échec transform/load, pas besoin de retélécharger
-

18/43 Nettoyage & anonymisation (T)

- Le seed=42 déterministe est ce qui a permis de repérer le bug 'Other' vs 'other' (diapo précédente sur les contraintes CHECK) — un run non déterministe l'aurait masqué de façon aléatoire selon les données tirées
 - Dérivation de ended_at : nécessaire car la source ne fournit que la durée, pas l'horodatage de fin
-

19/43 Chargement PostgreSQL (L)

- Le passage par une table de staging évite de verrouiller la table finale pendant tout le chargement — important une fois qu'Airflow orchestre ça en tâche de fond quotidienne
 - Historisation différenciée : biométrie = append-only (on veut la série complète), profils = upsert (on ne garde que l'état courant)
-

20/43 Orchestration Airflow

- Ce conflit de dépendances aurait cassé le webserver/scheduler si on l'avait raté — détecté via les warnings pip au build de l'image, avant tout déploiement
 - Repris par le rôle A en fin de projet, l'équipe ETL n'ayant pas le temps matériel de le finir
-

21/43 Tableau de bord Airflow

- Vue "DAGs" (page d'accueil) : un DAG enregistré, schedule @daily, 2 runs réussis / 1 échoué visible dans l'historique — pas nettoyé volontairement, cohérent avec le reste de la présentation qui assume les obstacles réels plutôt que de les cacher
 - L'échec en question vient d'un problème de permissions sur airflow/logs/ (le DagFileProcessorManager n'arrivait plus à écrire), corrigé et documenté dans le guide de déploiement
 - On va déclencher un run en direct dans quelques instants
-

22/43 05 • API REST

- Transition : les données sont en base, comment on les expose — rôle B, Florian
-

23/43 Structure

- La séparation models/ (ORM SQLAlchemy) vs schemas/ (Pydantic) n'est pas cosmétique : ça évite d'exposer par accident une colonne sensible comme hashed_password dans une réponse API
 - Les trois dépendances (CurrentUser/CurrentAdmin/CurrentPremiumUser) sont déclarées une fois et réutilisées comme simple type d'argument — pas de code de vérification dupliqué par route
 - On montrera /docs en direct pendant la démo
-

24/43 Endpoints principaux

- Distinction à faire à l'oral : les catalogues (food-items/exercises) sont alimentés par l'ETL, pas par les utilisateurs — lecture libre mais écriture réservée aux admins
 - data-quality duplique volontairement une partie de l'interface Gradio : utile pour l'intégration avec d'autres outils (scripts, monitoring), pas juste pour l'humain derrière l'écran
-

25/43 Documentation interactive (Swagger UI)

- Générée automatiquement par FastAPI depuis les schémas Pydantic — aucune doc à maintenir à la main
 - Accessible sur /docs, testable en direct (bouton Authorize + JWT) : c'est ce qu'on va montrer pendant la démo live plutôt que de rester sur cette capture
-

26/43 Abonnements & microservice IA

- C'est un pattern de préparation, pas une fonctionnalité IA livrée — être honnête là-dessus à l'oral
-

27/43 06 · Interface admin & qualité des données

- Ici on revient sur le périmètre du rôle A : l'outil de pilotage qualité, pas l'app utilisateur
-

28/43 Interface admin (Gradio)

- Filtres, tableau des anomalies, correction manuelle, export CSV/JSON — tout sur un écran
-

29/43 Accessibilité RGAA — démarche

- Ma propre première vérification manuelle était incomplète : j'avais raté les couleurs de texte info=/labels réellement utilisées — l'audit automatisé a révélé l'erreur, corrigé depuis
 - Leçon retenue : l'automatisation prime sur la relecture manuelle pour ce type de vérification
-

30/43 07 · Dashboard Metabase

- Complémentaire à l'interface admin : vue d'ensemble/tendances plutôt que résolution au cas par cas
-

31/43 Metabase

- Les vues apparaissent en table brute par défaut (comme n'importe quel outil BI sans modèle pré-configuré) — il faut construire des Questions dessus pour avoir des graphiques, fait pour la démo
-

32/43 08 • Dockerisation

- Comment tout ce qu'on vient de voir se lance en une seule commande
-

33/43 Huit services, une commande

- Choix assumé de s'écarter du patron du TP (CeleryExecutor) pour une raison concrète : mémoire
-

34/43 Point de vigilance — confirmé en conditions réelles

- Ce point illustre bien la démarche du projet : documenter le risque avant qu'il n'arrive, puis le confirmer/corriger avec des données réelles plutôt que de le laisser hypothétique
-

35/43 09 • Tests & CI/CD

- Ce qui donne confiance que tout ce qui a été montré fonctionne vraiment, pas juste "sur ma machine"
-

36/43 Suite de tests

- Les tests contre une vraie base (pas des mocks) sont un choix délibéré : les contraintes CHECK/FK portent une partie des règles métier, un mock ne les vérifie pas
 - Ça a concrètement attrapé un bug qu'on racontera dans le bilan
-

37/43 10 • Démonstration live

- Moment de bascule terminal/navigateur — prévenir le jury qu'on quitte les slides quelques minutes
-

38/43 Ce qu'on va montrer

- Basculer sur le terminal / navigateur maintenant
 - Rappel pour moi : ne pas relancer docker compose une fois le DAG déclenché (risque TRUNCATE)
-

39/43 11 • Bilan & conclusion

- Retour sur les slides après la démo : prendre du recul sur ce qui a été montré
-

40/43 Obstacles techniques rencontrés

- Le fil rouge : chaque fois, la vérification automatisée/réelle a trouvé ce qu'une vérification manuelle ou mockée avait raté — c'est la leçon principale du projet
-

41/43 Points positifs

- Insister ici : ces points positifs ne sont pas juste "ça a marché", ce sont des choix d'architecture délibérés pris tôt (Sprint 1) qui ont payé plus tard dans le projet
 - La discipline de revue a concrètement empêché plusieurs bugs (CHECK sex, source ETL divergente) d'arriver jusqu'en production — bon moment pour faire le lien avec le slide précédent
-

42/43 Conclusion

- Message de clôture : compromis assumé, pas un oubli
-

43/43 Merci

- Ouvrir les questions
- Garder sous la main : les chiffres clés (16 tables, 141 tests, 2851 users chargés en démo)